PRODUCTION-READY ARCHITECTURE BLUEPRINT

Enterprise RAG + Text-to-SQL System

Complete Architecture, Technology Stack & Implementation Guide for Scalable AI Retrieval & Autonomous Database Querying

DOCUMENT ROLE
Architecture & Engineering Specification

TARGET ARCHITECTURE
3-Layer Decoupled (Core / API / UI)

AUTHOR
Senior AI & RAG Solutions Architect

PUBLICATION DATE
September 2026

Strictly Confidential & Proprietary

Table of Contents

1. Executive Summary
2. System Architecture Overview
3. Technology Stack — The 10 Pillars of RAG
4. Recommended Stack Summary
5. Open-Source & Integration Compatibility
6. Project Structure
7. Data Flow & Pipeline Architecture
8. Automated Evaluation Pipeline
9. Phased Implementation Roadmap
10. Gap Analysis & Enhancements
11. Installation & Quick Start
12. Key Recommendations

1. Executive Summary

This document serves as the **complete technical blueprint** for building an Enterprise-grade Retrieval-Augmented Generation (RAG) system with Text-to-SQL capabilities. The system is designed to:

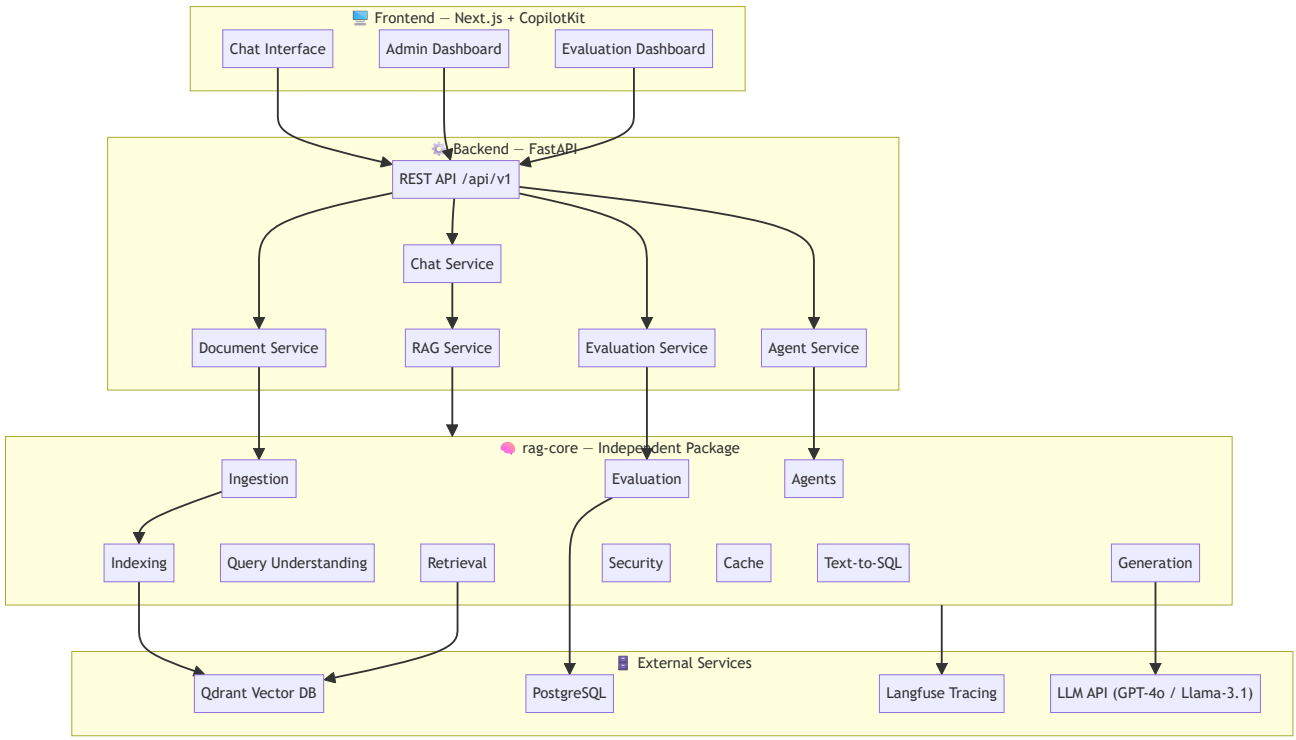
- **Answer questions** from indexed documents using semantic search and LLM generation
- **Query databases** using natural language via Text-to-SQL
- **Orchestrate multiple tools** through an intelligent LangGraph agent
- **Continuously evaluate quality** with RAGAS metrics and automated CI/CD pipelines
- **Scale to production** with Docker, Kubernetes, and enterprise security

> ⭐ **IMPORTANT**
>
> This architecture follows a **3-layer separation** pattern: - `rag-core` — Reusable, framework-independent RAG package - `backend` — FastAPI API server - `frontend` — Next.js + CopilotKit chat interface

2. System Architecture Overview

High-Level Architecture Diagram

ARCHITECTURE DIAGRAM — High-Level System Architecture



Three-Layer Design Philosophy

Layer	Package	Purpose	Independence
rag-core	Python package	All RAG logic — ingestion, retrieval, generation, evaluation, agents	✅ Can be used in **any** Python project without FastAPI or Next.js
backend	FastAPI app	API endpoints, authentication, input validation, calls rag-core	Depends on rag-core
frontend	Next.js app	Chat UI with CopilotKit, admin dashboard, evaluation dashboard	Depends on backend API

> 💡 **TIP**
>
> The `rag-core` package is **completely independent** from the web application. You can import it into a Jupyter notebook, a CLI script, or any other Python project.

3. Technology Stack — The 10 Pillars of RAG

Every production RAG system must address these 10 critical areas. For each pillar, we list the available technologies, our recommended choice, and the reasoning.

Pillar 1: Project Setup & Core Stack

Component	Options	✅ Recommended	Why
Language	Python	**Python**	Industry standard for the LLM ecosystem
RAG Framework	LlamaIndex, LangChain, Haystack	**LlamaIndex**	Specialized for indexing, search, and post-retrieval processing
Agent Framework	LangChain, LlamaIndex, CrewAI	**LangChain + LangGraph**	Superior for complex agents, tool management, and orchestration
Config Management	Pydantic, Hydra	**Pydantic**	Type-safe settings validation and code structuring

> ℹ️ **NOTE**
>
> **LlamaIndex** handles the RAG pipeline (ingestion → retrieval → generation). **LangChain/LangGraph** handles agent orchestration and tool management. They are **officially integrated** and work together without conflicts.

Pillar 2: Ingestion & Indexing

This pillar handles loading documents, splitting them into chunks, generating embeddings, and storing vectors.

Component	Options	✅ Recommended	Why
Document Loading	LlamaHub, LangChain Loaders	**LlamaHub**	Supports 300+ data sources (PDF, DB, API, S3, etc.)
Complex PDF Parsing	LlamaParse, Unstructured	**LlamaParse**	Best for extracting tables and text from scanned/complex PDFs
Chunking Strategy	SentenceSplitter, SemanticSplitter	**SemanticSplitter**	Splits by semantic meaning, not just character count
Embedding Model	OpenAI, BGE-M3, Cohere	**BGE-M3**	Multilingual, free, runs locally, excellent RAG performance
Vector Database	Chroma, Qdrant, Milvus, pgvector	**Qdrant** (start) → **Milvus** (scale)	Qdrant: great balance of performance + metadata filtering. Milvus: for millions+ documents

Chunking Configuration

```
SemanticSplitterNodeParser:
  buffer_size: 1
  breakpoint_percentile_threshold: 95
```

Embedding Model Details — BGE-M3

- **License**: MIT (fully open-source)
- **Multilingual**: Supports Persian, English, and 100+ languages
- **Local execution**: Via `HuggingFaceEmbedding` or inference servers (vLLM, Xinference)
- **Install**: `pip install llama-index-embeddings-huggingface`

Pillar 3: Query Understanding & Transformation

Transforms user queries into optimized forms before retrieval.

Component	Options	✅ Recommended	Why
Query Rewriting	LlamaIndex QueryTransform, LangChain	**LlamaIndex QueryTransform**	Built-in support for rewriting, expansion, and decomposition
Query Routing	LlamaIndex Router, LangChain Router	**LlamaIndex Router**	Routes queries to specialized indexes or pipelines
Intent Detection	LangChain, Semantic Router	**LangChain**	Classifies user intent to select the right tool or path

Supported Transformations

- **Query Rewriting** — Rephrase ambiguous queries for clarity
- **Query Expansion** — Add synonyms and related terms
- **HyDE** (Hypothetical Document Embedding) — Generate a hypothetical answer, embed it, search with it
- **Step-back Prompting** — Ask a broader question first to establish context
- **Sub-question Decomposition** — Break complex questions into simpler sub-queries

> ℹ️ **NOTE**
>
> All query transformation features are built into the LlamaIndex core — no separate packages needed.

Pillar 4: Retrieval & Re-ranking

The core retrieval engine that finds the most relevant documents.

Component	Options	✅ Recommended	Why
Search Strategy	Vector Search, Hybrid Search	**Hybrid Search**	Combines semantic (vector) + keyword (BM25) for higher accuracy
Re-ranking	Cohere Rerank, BGE-Reranker	**BGE-Reranker v2 M3** (start)	Open-source, free, Apache 2.0 license
Metadata Filtering	Qdrant, Weaviate	**Qdrant**	Powerful payload filtering by date, author, category, access level

Re-ranker Model Comparison

Model	ELO Score	License	Size	Best For
Zerank 1 Small	1359	Apache 2.0	Small	Best quality/freedom balance — self-host easily
BGE Reranker v2 M3	1327	Apache 2.0	Medium	Lightweight, fast — great for limited hardware
Qwen3 Reranker 8B	1473	Open	8B	Highest accuracy if you can run an 8B model

Hybrid Search Configuration

```
alpha: 0.7    # 70% semantic (vector) + 30% keyword (BM25)
top_k: 5-10   # Number of documents to retrieve
```

Pillar 5: Generation & Prompt Engineering

Generate answers from retrieved context using LLMs.

Component	Options	✅ Recommended	Why
LLM	GPT-4o, Claude-3.5, Llama-3.1	**GPT-4o** (quality) / **Llama-3.1** (privacy)	GPT-4o for strong reasoning; Llama-3.1 for on-premise, full data control
Prompt Templates	LangChain PromptTemplate	**LangChain PromptTemplate**	Chain-of-Thought, structured prompts, easy templating
Structured Output	LangChain Output Parsers	**PydanticOutputParser**	Get reliable JSON from LLM with validation

RAG Prompt Strategy

```
System Prompt: "Answer ONLY based on the provided context.
                If the information is insufficient, say 'I don't know'.
                Always cite which document(s) your answer is based on."
```

Response Synthesis Strategies

- **Compact** — Merge all context into one prompt (fast, good for short contexts)
- **Refine** — Iteratively refine the answer document by document (better for many docs)
- **Tree Summarize** — Hierarchical summarization for very long contexts

---

Pillar 6: Agent & Orchestration

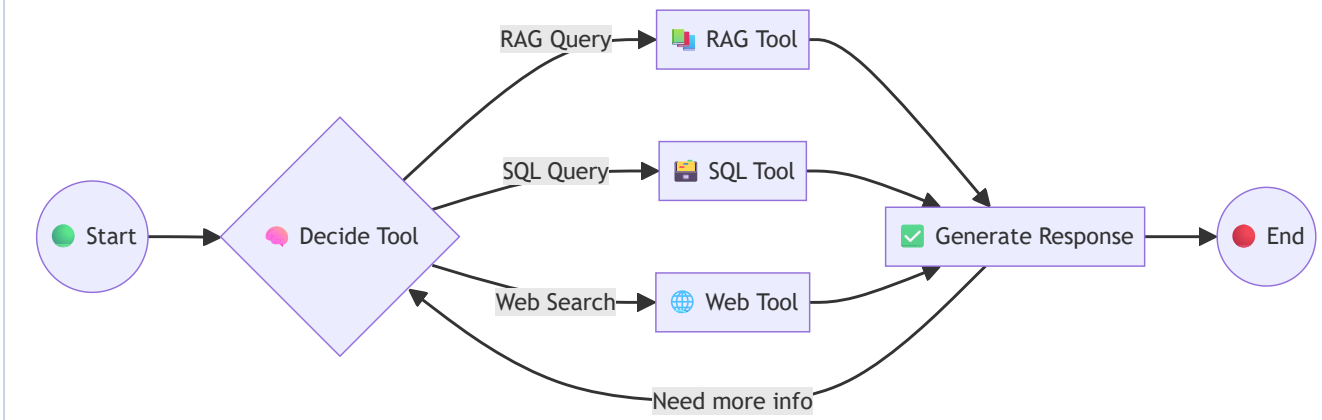
Build intelligent agents that can use multiple tools (RAG, SQL, web search).

Component	Options	✅ Recommended	Why
Agent Structure	LangChain Agent, LangGraph	**LangGraph**	Builds complex, non-linear, conditional multi-step workflows
Tools	LangChain Tools	**LangChain Tools**	Wrap RAG QueryEngine as a Tool callable by the agent
Memory	LangChain Memory, LangGraph Checkpointing	**LangGraph Checkpointing**	Persist conversation state across complex multi-step tasks

Agent Architecture (LangGraph StateGraph)

ARCHITECTURE DIAGRAM

Agent Decision & Tool Routing Flow



> ⭐ **IMPORTANT**
>
> **Text-to-SQL should be a SEPARATE Tool in the Agent**, not merged with RAG. This keeps concerns separated and allows independent optimization.

Pillar 7: RAG Evaluation — THE CRITICAL PILLAR ⚠️

Without evaluation, you cannot measure improvement. This is the **most important** pillar.

Component	Options	✅ Recommended	Why
Evaluation Framework	RAGAS, DeepEval, TruLens	**RAGAS**	Industry standard for RAG evaluation (Apache 2.0)
Key Metrics	Faithfulness, Relevancy, Precision, Recall	**All four RAGAS metrics**	Comprehensively measure both retrieval and generation quality
Test Dataset	Golden Test Set	**Golden Test Set**	Curated set of 20-50 ideal Q&A pairs for regular evaluation

The Four Core RAGAS Metrics

Metric	Threshold	What It Measures
Faithfulness	≥ 0.80	Is the answer grounded in the retrieved context? (No hallucination)
Answer Relevancy	≥ 0.75	Is the answer relevant to the user's question?
Context Precision	≥ 0.75	Are the retrieved documents actually relevant?
Context Recall	≥ 0.70	Does retrieval cover ALL necessary information?

> 🛑 **CAUTION**
>
> Without RAGAS, you **cannot know** if your changes actually improved the system. Start evaluation from Day 1.

Pillar 8: Observability & Monitoring

Track every operation in the RAG pipeline for debugging and optimization.

Component	Options	✅ Recommended	Why
Tracing	Langfuse, LangSmith	**Langfuse**	Open-source (MIT), self-hostable, deep tracing for LLM + RAG
Logging	structlog, Python logging	**structlog**	Structured JSON logging with search capabilities
Metrics	Prometheus, Grafana	**Prometheus + Grafana**	Monitor latency, throughput, error rates in production

What Langfuse Traces

- LLM API calls (tokens, latency, cost)
- Embedding operations
- Retrieval queries and results
- Agent decision paths
- Full pipeline end-to-end traces

Pillar 9: Enterprise Security & Access Control

Production-grade security measures.

Component	Options	✅ Recommended	Why
Access Control	RBAC, ABAC, Qdrant filters	**Qdrant Payload Filtering**	Apply ACL at the **retrieval level**, not just at the prompt
PII Redaction	Presidio, SpaCy	**Presidio**	Microsoft's tool for detecting/removing PII (MIT license)
Audit Logging	Custom	**Structured Logging**	Log all access and operations for compliance

> ⚠️ **WARNING**
>
> Access control must be enforced at the **retrieval level** (filtering which documents a user can see), not just at the prompt level. Prompt-level security is easily bypassed.

Pillar 10: Deployment & Scaling

Production deployment infrastructure.

Component	Options	✅ Recommended	Why
API Framework	FastAPI	**FastAPI**	Fast, async, ideal for RAG services
Containerization	Docker, Podman	**Docker**	Package app + dependencies together
Orchestration	Kubernetes	**Kubernetes**	Auto-scaling, load balancing, container management
Scalable Vector DB	Milvus, Qdrant (clustered)	**Milvus**	When documents reach millions/billions

4. Recommended Stack Summary

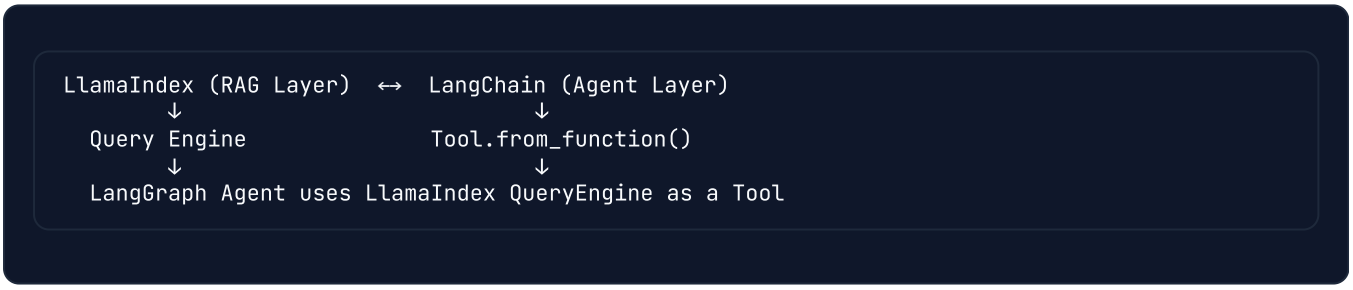
Category	Selection
RAG Framework	**LlamaIndex**
Agent Framework	**LangChain + LangGraph**
Vector Database	**Qdrant** (start) → **Milvus** (scale)
Embedding Model	**BGE-M3** (BAAI/bge-m3)
LLM	**GPT-4o** (quality) / **Llama-3.1** (privacy)
Re-ranker	**BGE-Reranker v2 M3** (start) → **Zerank 1 Small** (production)
Evaluation	**RAGAS**
Observability	**Langfuse**
Backend API	**FastAPI**
Frontend	**Next.js 14+ App Router + CopilotKit**
PII Protection	**Presidio**
Database	**PostgreSQL** (eval results, metadata)
CI/CD	**GitHub Actions**

5. Open-Source & Integration Compatibility

All recommended technologies are open-source and work together without conflicts.

Technology	Open-Source?	License	Conflicts?
LlamaIndex	✅ Yes	MIT	None — officially integrates with LangChain
LangChain + LangGraph	✅ Yes	MIT	None — LangGraph is built on LangChain
BGE-M3	✅ Yes	MIT	None — works via HuggingFaceEmbedding
Qdrant	✅ Yes	Apache 2.0	None — separate package `llama-index-vector-stores-qdrant`
RAGAS	✅ Yes	Apache 2.0	None — works with both LlamaIndex and LangChain
Langfuse	✅ Yes	MIT	None — deep integration with both frameworks
Presidio	✅ Yes	MIT	None — standalone library, used as pre-processing step
FastAPI	✅ Yes	MIT	N/A
Milvus	✅ Yes	Apache 2.0	None — separate package `llama-index-vector-stores-milvus`

Key Integration Architecture



> ℹ️ **NOTE**
>
> LlamaIndex uses a **modular architecture** — each integration lives in a separate package (e.g., `llama-index-vector-stores-qdrant`). This avoids installing unnecessary dependencies.

6. Project Structure

Complete Directory Tree

```
project-root/
├── frontend/                          # Next.js 14+ App Router + CopilotKit
│   ├── src/
│   │   ├── app/
│   │   │   ├── layout.tsx             # Root layout: providers, metadata, global styles
│   │   │   ├── page.tsx               # Home page / landing
│   │   │   ├── chat/
│   │   │   │   └── page.tsx           # Main RAG chat interface
│   │   │   └── dashboard/
│   │   │       ├── evaluation/
│   │   │       │   ├── page.tsx       # Evaluation metrics dashboard
│   │   │       │   └── components/
│   │   │       │       ├── MetricsChart.tsx    # RAGAS metrics trend chart
│   │   │       │       ├── EvalRunTable.tsx    # Historical eval runs table
│   │   │       │       └── QualityGateStatus.tsx  # Pass/fail badge
│   │   │       ├── documents/
│   │   │       │   └── page.tsx       # Document upload & management
│   │   │       └── settings/
│   │   │           └── page.tsx       # RAG pipeline configuration UI
│   │   ├── components/
│   │   │   ├── chat/
│   │   │   │   ├── ChatInterface.tsx  # Chat message list + input
│   │   │   │   └── CopilotProvider.tsx  # CopilotKit context wrapper
│   │   │   └── ui/                    # Reusable UI components (Button, Card, etc.)
│   │   ├── lib/
│   │   │   ├── api/
│   │   │   │   ├── client.ts          # Axios/fetch HTTP client for backend
│   │   │   │   └── endpoints.ts       # Typed API endpoint wrappers
│   │   │   ├── copilot/
│   │   │   │   ├── actions.ts         # CopilotKit action definitions
│   │   │   │   └── config.ts          # CopilotKit runtime configuration
│   │   │   └── utils/                 # Shared utility functions
│   │   ├── hooks/                     # Custom React hooks
│   │   ├── styles/                    # Global CSS and theme
│   │   └── types/                     # Shared TypeScript types
│   ├── public/                        # Static assets
│   ├── package.json
│   ├── next.config.js                 # API proxy, output config
│   ├── tsconfig.json                  # TypeScript compiler settings
│   └── Dockerfile                     # Multi-stage production build
│
├── backend/                           # FastAPI API Server
│   ├── app/
│   │   ├── main.py                    # App entry: CORS, routers, lifecycle
│   │   ├── api/
│   │   │   ├── v1/
│   │   │   │   ├── endpoints/
│   │   │   │   │   ├── chat.py        # POST /chat — RAG queries + streaming
│   │   │   │   │   ├── rag.py         # RAG-specific endpoints (retrieve, index)
│   │   │   │   │   ├── documents.py   # Document upload, list, delete
│   │   │   │   │   ├── evaluation.py  # Trigger eval, view results, manage baseline
│   │   │   │   │   └── health.py      # System health check
│   │   │   │   └── router.py          # Aggregate all v1 routers
│   │   │   └── deps.py                # Dependency injection (DB session, auth, etc.)
│   │   ├── core/
│   │   │   ├── config.py              # Pydantic-settings: environment config
│   │   │   ├── logging.py             # structlog: structured JSON logging
│   │   │   ├── security.py            # JWT, password hashing, API keys
│   │   │   └── exceptions.py          # Custom exceptions + global handlers
│   │   ├── services/
│   │   │   ├── chat_service.py        # Chat logic: routing, conversation mgmt
│   │   │   ├── rag_service.py         # Bridge to rag-core RAG pipeline
```

```
│   │   │   ├── document_service.py      # Document lifecycle: upload → index
│   │   │   ├── agent_service.py         # LangGraph agent orchestration
│   │   │   └── evaluation_service.py    # Bridge to rag-core evaluation module
│   │   ├── schemas/
│   │   │   ├── chat.py                  # Request/response models for chat
│   │   │   ├── rag.py                   # Models for RAG endpoints
│   │   │   ├── document.py              # Models for document management
│   │   │   └── evaluation.py            # Models for evaluation endpoints
│   │   ├── db/
│   │   │   ├── base.py                  # SQLAlchemy Base + mixins
│   │   │   ├── session.py               # Async session factory
│   │   │   └── models.py                # ORM models: eval_runs, documents, conversations
│   │   └── utils/                       # Backend utility helpers
│   ├── alembic/                         # 🔧 [GAP FIX] Database migrations
│   │   ├── alembic.ini
│   │   ├── env.py
│   │   └── versions/                    # Auto-generated migration scripts
│   ├── tests/
│   │   ├── test_chat.py
│   │   ├── test_rag.py
│   │   └── test_evaluation.py
│   ├── pyproject.toml
│   ├── requirements.txt
│   └── Dockerfile
│
├── rag-core/                            # 🧠 Independent, Modular RAG Package
│   ├── src/
│   │   └── rag_core/
│   │       ├── __init__.py
│   │       ├── config.py                # Central RAG pipeline configuration
│   │       │
│   │       ├── ingestion/               # 📥 Document Loading & Parsing
│   │       │   ├── loaders.py           #    LlamaHub/SimpleDirectoryReader wrappers
│   │       │   ├── parsers.py           #    Complex PDF parsing (LlamaParse/Unstructured)
│   │       │   └── splitters.py         #    Semantic + sentence chunking
│   │       │
│   │       ├── indexing/                # 💾 Embedding & Vector Storage
│   │       │   ├── embeddings.py        #    BGE-M3 model initialization
│   │       │   ├── vector_store.py      #    Qdrant/Milvus abstraction
│   │       │   └── indexer.py           #    Full ingestion→index orchestrator
│   │       │
│   │       ├── query/                   # 🔍 Query Processing
│   │       │   ├── transformation.py    #    Rewriting, expansion, HyDE
│   │       │   ├── routing.py           #    Route to specialized indexes
│   │       │   └── understanding.py     #    Intent detection, decomposition
│   │       │
│   │       ├── retrieval/               # 📊 Search & Re-ranking
│   │       │   ├── retriever.py         #    Core vector retriever + metadata filtering
│   │       │   ├── hybrid.py            #    Hybrid search (vector + BM25)
│   │       │   └── reranker.py          #    BGE-Reranker / Cohere postprocessor
│   │       │
│   │       ├── generation/              # 💬 LLM Response Generation
│   │       │   ├── llm.py               #    LLM client (GPT-4o / Llama-3.1)
│   │       │   ├── prompts.py           #    RAG prompt templates (CoT, system)
│   │       │   └── synthesizer.py       #    Response synthesis (compact/refine/tree)
│   │       │
│   │       ├── postprocessing/          # ✨ Output Enhancement
│   │       │   ├── compression.py       #    Context compression, LongContextReorder
│   │       │   └── enhancement.py       #    Source citations, formatting, guardrails
│   │       │
│   │       ├── evaluation/              # 📏 RAG Evaluation (RAGAS)
│   │       │   ├── __init__.py
│   │       │   ├── run_eval.py          #    CLI entry point for evaluation
│   │       │   ├── harness.py           #    EvaluationHarness: runs RAGAS scoring
│   │       │   ├── metrics.py           #    Custom domain-specific metrics
│   │       │   ├── config.py            #    Thresholds and evaluation settings
│   │       │   ├── quality_gate.py      #    Pass/fail logic based on thresholds
│   │       │   ├── baseline_manager.py  #    Save/load/compare baseline scores
│   │       │   ├── report_generator.py  #    HTML + JSON report generation
│   │       │   └── datasets/
│   │       │       ├── golden_set.jsonl #    Versioned Q&A pairs (50+ curated)
│   │       │       └── load_dataset.py  #    Load and validate golden set
```

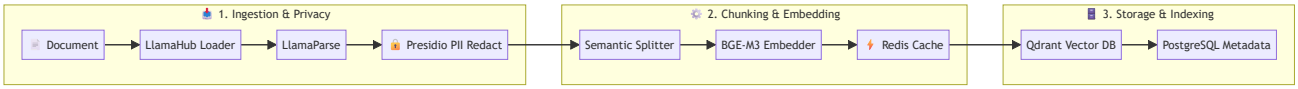

baselines/
baseline.json # Approved scores snapshot
reports/ # Generated evaluation reports
observability/ # Tracing & Logging
langfuse.py # Langfuse client + callback handlers
logging.py # structlog configuration
agents/ # LangGraph Agent
tools/
rag_tool.py # RAG QueryEngine as LangChain Tool
sql_tool.py # Text-to-SQL as LangChain Tool
web_tool.py # Web search fallback Tool
agent.py # LangGraph StateGraph orchestrator
security/ # [GAP FIX] Enterprise Security
pii_redactor.py # Presidio PII detection & redaction
acl_filter.py # Qdrant payload-based access control
audit.py # Structured audit logging
cache/ # [GAP FIX] Performance Caching
embedding_cache.py # Cache embeddings by content hash
llm_cache.py # Cache LLM responses (TTL-based)
text_to_sql/ # [GAP FIX] Text-to-SQL Module
chain.py # LangChain SQL generation chain
schema_loader.py # Database schema introspection
validator.py # SQL safety validation (read-only)
prompts.py # SQL-specific prompt templates
utils/ # Shared helpers
tests/
test_ingestion.py
test_retrieval.py
test_evaluation.py
test_quality_gate.py
pyproject.toml
.github/ # CI/CD Workflows
workflows/
rag-eval.yml # Auto eval on PR (when RAG files change)
scheduled-eval.yml # Weekly scheduled eval (drift detection)
deploy.yml # Build + deploy to production
docker-compose.yml # All services: frontend, backend, qdrant, langfus
.env.example # Environment variable template
.gitignore
Makefile # Common commands: make dev, build, test, eval
README.md

7. Data Flow & Pipeline Architecture

Document Ingestion Flow

ARCHITECTURE DIAGRAM

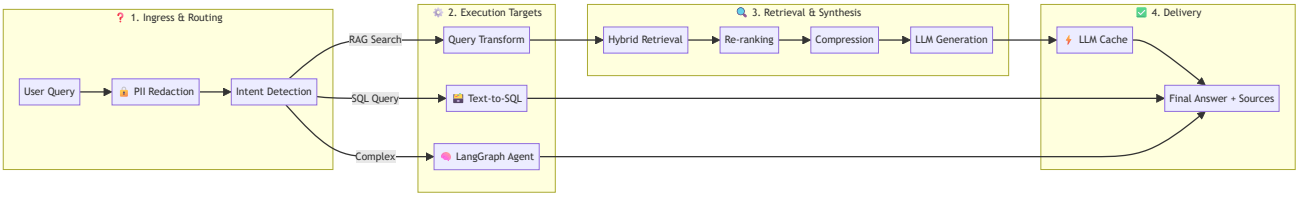
End-to-End Document Ingestion Pipeline



Query Processing Flow

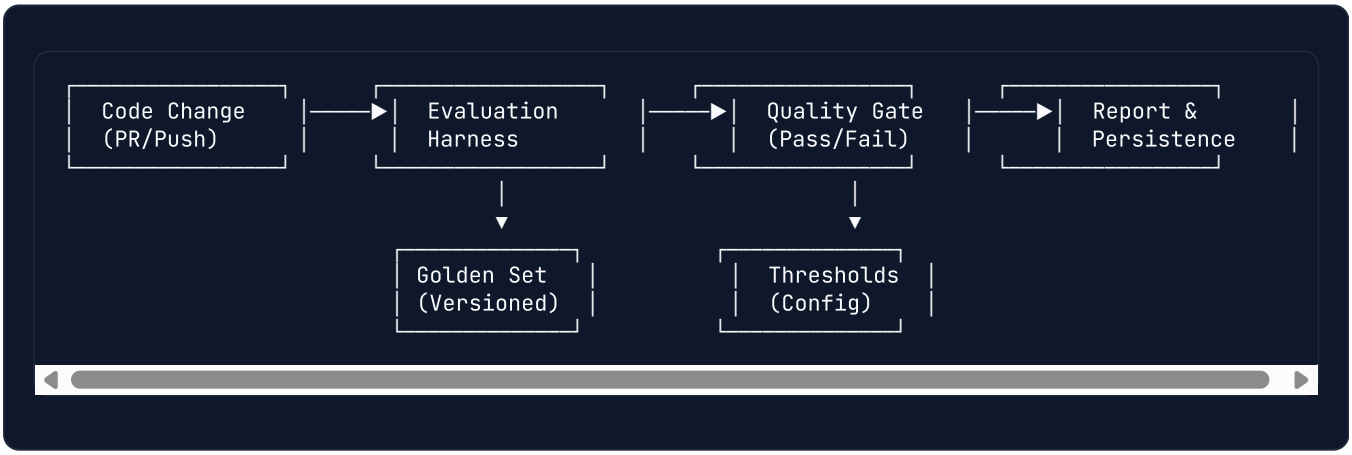
ARCHITECTURE DIAGRAM

Query Processing & Retrieval Pipeline



8. Automated Evaluation Pipeline

Pipeline Architecture



Evaluation Triggers

Not every code change requires a full evaluation. Only RAG-relevant changes trigger evaluation:

Change Type	Files Affected	Evaluation Type
Prompt changes	`prompts/*.py`, `*.prompt`	Full End-to-End evaluation
Model/config changes	`config/llm.yaml`, `config/model.py`	Full End-to-End evaluation
Retrieval config	`chunking.py`, `embedding.py`	Retrieval + Full evaluation
Other changes	Application code, dependencies	⏭️ Skip (save cost/time)

Quality Gate Thresholds

Metric	Minimum	If Below
Faithfulness	≥ 0.80	❌ FAIL — answers may be hallucinated
Answer Relevancy	≥ 0.75	❌ FAIL — answers are off-topic
Context Precision	≥ 0.75	❌ FAIL — irrelevant docs retrieved
Context Recall	≥ 0.70	❌ FAIL — missing critical information

Evaluation Integration Points

Integration	Where	What It Does
CI/CD	`.github/workflows/rag-eval.yml`	Auto-run on PR, block merge if quality drops
Backend API	`/api/v1/evaluation/run`	Trigger manual evaluation, view results
Frontend	`/dashboard/evaluation`	Visualize metrics trends, quality gate status
Scheduled	`.github/workflows/scheduled-eval.yml`	Detect gradual quality drift (weekly)
PostgreSQL	`eval_runs` table	Store all results for historical analysis

Continuous Evaluation Checklist

- ☐ Build a Golden Set with at least 50 curated Q&A pairs (version in Git)
- ☐ Implement EvaluationHarness in `rag-core/evaluation/`
- ☐ Define Quality Gate thresholds (Faithfulness ≥ 0.80, etc.)
- ☐ Save Baseline from a confirmed-good version
- ☐ Build GitHub Actions workflow for automatic PR evaluation
- ☐ Store results in PostgreSQL for trend analysis
- ☐ Build Frontend dashboard for evaluation history visualization
- ☐ Enable weekly scheduled evaluation for drift detection
- ☐ Set up Slack/Email alerts for quality degradation

Supplementary Evaluation Tools

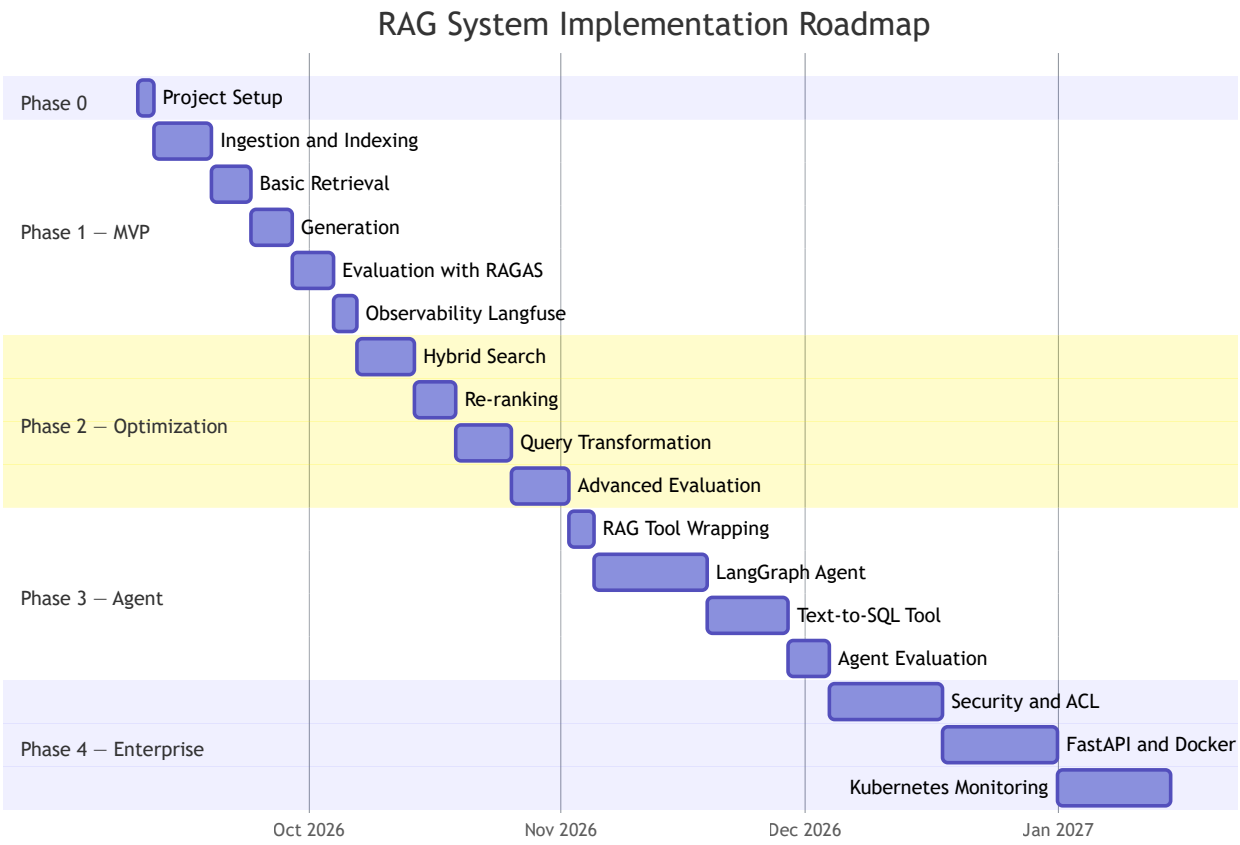
Tool	Purpose	Install
RAGAS	Primary evaluation engine (industry standard)	`pip install ragas`
RagaliQ	RAG testing with pytest integration	`pip install ragaliq`
rag-ci	Regression testing + GitHub Actions	`uvx rag-ci`
DeepEval	LLM-as-judge evaluation	`pip install deepeval`
Langfuse	Tracing and observability	`pip install langfuse`

9. Phased Implementation Roadmap

Overview Timeline

ARCHITECTURE DIAGRAM

Implementation Roadmap Timeline



Phase 0: Project Setup — 1-2 Days

Goal: Prepare the development environment and project skeleton.

Tasks: 1. Create Python virtual environment 2. Install core packages: `bash pip install llama-index langchain langgraph langfuse ragas qdrant-client sentence-transformers` 3. Set up `config.py` with environment settings 4. Initialize project directory structure

Output: ✅ Ready development environment with clear project structure

Phase 1: MVP — 2-4 Weeks

Goal: Build a basic working RAG that answers questions and is measurable.

Components: Ingestion, Basic Retrieval, Generation, Evaluation, Observability

Step	Task	Details
1.1	**Ingestion & Indexing**	Load with `SimpleDirectoryReader`, chunk with `SemanticSplitter`, embed with `BGE-M3`, store in Qdrant
1.2	**Basic Retrieval**	Vector similarity search with `top_k=5-10`, metadata filtering
1.3	**Generation**	Start with `Llama-3.1-8B` via Ollama (zero cost), upgrade to GPT-4o if needed
1.4	**Evaluation**	Build Golden Set (20-50 Q&A pairs), compute 4 RAGAS metrics
1.5	**Observability**	Integrate Langfuse to trace all pipeline steps

Output: ✅ A working RAG with evaluation and tracing

Phase 2: Optimization — 4-6 Weeks

Goal: Significantly improve retrieval accuracy and answer quality.

Step	Task	Details
2.1	**Hybrid Search**	BM25 + Vector in Qdrant, `alpha=0.7`
2.2	**Re-ranking**	Add `BGE-Reranker v2 M3` postprocessor
2.3	**Query Understanding**	Rewriting, expansion, SubQuestion decomposition, routing
2.4	**Advanced Evaluation**	Analyze weak metrics, optimize chunk size / top_k / model

Output: ✅ Measurably improved accuracy

Phase 3: Agent & Orchestration — 4-6 Weeks

Goal: Build an intelligent multi-tool agent.

Step	Task	Details
3.1	**Tool Wrapping**	Wrap QueryEngine as `LangChain Tool`
3.2	**LangGraph Agent**	StateGraph with decision, tool, and response nodes
3.3	**Text-to-SQL**	`create_sql_query_chain` + validation as separate Tool
3.4	**Agent Evaluation**	RAGAS agentic metrics, trace decisions in Langfuse

Output: ✅ Intelligent assistant coordinating multiple tools

Phase 4: Enterprise — 6-12 Weeks

Goal: Production-ready with security, scalability, and monitoring.

Step	Task	Details
4.1	**Security**	PII Redaction (Presidio), ACL (Qdrant filters), Audit Logging
4.2	**Deployment**	FastAPI endpoints, Docker, Kubernetes, Milvus migration
4.3	**Monitoring**	Prometheus + Grafana, Langfuse alerts for quality drops

Output: ✅ Enterprise-grade RAG with full security and monitoring

---

Phase Summary

Phase	Timeline	Key Output
Phase 0	1-2 days	Ready environment
Phase 1 (MVP)	2-4 weeks	Working RAG with metrics
Phase 2 (Optimize)	4-6 weeks	High accuracy
Phase 3 (Agent)	4-6 weeks	Multi-tool assistant
Phase 4 (Enterprise)	6-12 weeks	Production-ready

---

10. Gap Analysis & Enhancements

The following gaps were identified in the original architecture and have been addressed:

🔒 Gap 1: Missing Security Module

Problem: The plan mentions Presidio PII and Qdrant ACL but had no dedicated `security/` module.

Fix: Added `rag-core/src/rag_core/security/` with:

File	Purpose
`pii_redactor.py`	Detect and remove PII using Microsoft Presidio
`acl_filter.py`	Retrieval-level access control via Qdrant payload filters
`audit.py`	Structured audit logging for compliance

⚡ Gap 2: Missing Caching Layer

Problem: No caching strategy — critical for cost reduction and latency improvement.

Fix: Added `rag-core/src/rag_core/cache/` with:

File	Purpose
`embedding_cache.py`	Cache embeddings by content hash (avoid re-embedding)
`llm_cache.py`	Cache LLM responses by query+context hash (TTL-based)

🗃️ Gap 3: Thin Text-to-SQL Implementation

Problem: Text-to-SQL was only a tool wrapper. Production SQL generation needs schema management, validation, and specialized prompts.

Fix: Added `rag-core/src/rag_core/text_to_sql/` with:

File	Purpose
`chain.py`	LangChain SQL generation chain
`schema_loader.py`	Database schema introspection for prompt context
`validator.py`	SQL safety — block DROP/DELETE, enforce SELECT-only
`prompts.py`	SQL-specific prompt templates with few-shot examples

🗄️ Gap 4: Missing Database Migrations

Problem: Backend has PostgreSQL models but no migration tool.

Fix: Added `backend/alembic/` with:

File	Purpose
`alembic.ini`	Alembic configuration
`env.py`	Migration runtime — auto-detect model changes
`versions/`	Auto-generated migration scripts

---

11. Installation & Quick Start

Prerequisites

- Python ≥ 3.11

- Node.js ≥ 18
- Docker & Docker Compose

Install Core Dependencies

```
# Core RAG frameworks
pip install llama-index langchain

# Integration packages (LlamaIndex modular architecture)
pip install llama-index-vector-stores-qdrant
pip install llama-index-embeddings-huggingface
pip install llama-index-postprocessor-cohere-rerank

# Agent, evaluation, observability, security
pip install langgraph langfuse ragas presidio-analyzer presidio-anonymizer
```

Start All Services

```
# Clone and configure
cp .env.example .env   # Fill in your API keys and settings

# Start everything
make dev               # Runs docker-compose with all services

# Or manually:
docker-compose up -d    # Start Qdrant, PostgreSQL, Langfuse
cd backend && uvicorn app.main:app --reload    # Start API
cd frontend && npm run dev                      # Start UI
```

12. Key Recommendations

> ⭐ **IMPORTANT**
>
> **The 5 Golden Rules of RAG Development**

1. 🪚 **Start evaluation from Day 1.** Without RAGAS, you cannot know if your changes actually improved the system. Every optimization must be measurable.
2. 🧱 **Always build an MVP first, then optimize.** Don't try to implement all 10 pillars simultaneously. Follow the phased roadmap.
3. 🆓 **Use BGE-M3 and BGE-Reranker v2 M3 to start.** They are open-source, free, and provide excellent performance for initial development.
4. 👁️ **Add Langfuse from the very beginning.** When issues arise (and they will), having traces from day one makes debugging dramatically easier.

5. 🔀 **Keep Text-to-SQL as a SEPARATE Tool in the Agent.** Don't merge it with RAG. The Agent decides when to use RAG vs SQL based on the query.

Priority Ranking of All 10 Pillars

Priority	Pillar	Importance	When to Start
1	RAG Evaluation	🔴 Critical	Day 1
2	Ingestion & Indexing	🔴 Critical	Phase 1 (MVP)
3	Retrieval & Re-ranking	🔴 Critical	Phase 1 (MVP)
4	Generation & Prompts	🔴 Critical	Phase 1 (MVP)
5	Observability	🟠 Very Important	Phase 1 (MVP)
6	Query Understanding	🟡 Important	Phase 2 (Optimize)
7	Agent & Orchestration	🟡 Important	Phase 3 (Agent)
8	Security & ACL	🟡 Important	Phase 4 (Enterprise)
9	Deployment & Scaling	🟡 Important	Phase 4 (Enterprise)
10	Project Setup	⚪ Prerequisite	Phase 0

This document is a living reference. Update it as the project evolves through each phase.

Last updated: September 2026